	them is added), they will be part of the same disjoint set. 3) If there is a chain of elements x1, x2, …, xn such that x1 has been merged to x2, x2 has been merged with x3 and so on, then x1 and xn will be in the same partition. 4) If two elements are not in the same partition, then there is no other element belonging to both element's disjoint sets.

First, we'd like, obviously, to initialize our instance on construction. Without any loss of generality, we can restrict to the case where the *Universe* `U`, i.e. the set of all possible elements, is known in advance and static. We also assume that initially every element is in its own partition: workarounds to support violations to these assumptions are, anyway, easily achievable, by making wise use of dynamic arrays and the class' very own methods.

Finally, throughout this chapter we assume the elements of our *Universe* `U` are the integers between `0` and `n-1`. This is not a real restriction, as we can easily associate an index to each of the actual elements of `U`.

Initialization, therefore, just takes care of allocation the basic fields needed by the class, and assigns each element into its own partition.

The `findPartition` method, when called on an element `x` of `U`, will return the partition to which `x` belongs to. This output might not be meaningful outside of the instance of our data structure: think of this method mostly as a *protected*